

Stochastic Traffic Network Routing

A Probabilistic Load Minimization Approach

Yichen Ren Yixuan Fu

Advised by Prof. Raymond Chi Wing Wong

Department of Computer Science
The Hong Kong University of Science and Technology

COMP 4981 FYP Presentation

April 2026

Outline

- 1 Introduction
- 2 Methodology
- 3 Implementation & Results
- 4 Discussion & Conclusion

Outline

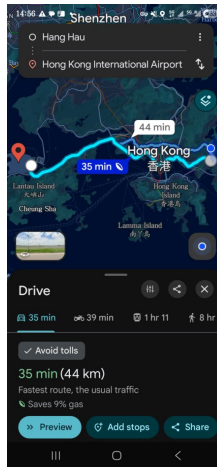
- 1 Introduction
- 2 Methodology
- 3 Implementation & Results
- 4 Discussion & Conclusion

Why Classical Routing Falls Short

- Dijkstra / A* assume **fixed** edge weights
- Real travel times are **random** — weather, accidents, demand changes
- Everyone with the same origin–destination gets the **same route** $\Rightarrow$ more congestion

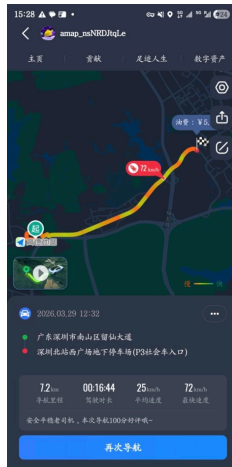
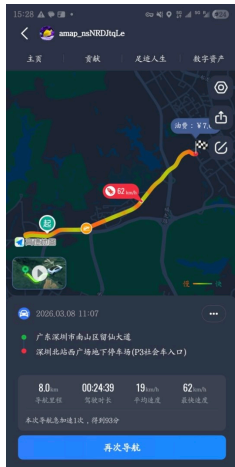
Can we do better?

Problem 1: Everyone Gets the Same Route



Same query $\Rightarrow$ same answer All
drivers pile onto the same road
Routing itself creates
congestion

Problem 2: Travel Time Is Unpredictable



Same route, similar departure time

Yet travel times differ a lot

⇒ Travel time is **stochastic**

We need to model it as a **distribution**, not a single number

What We Want to Achieve

Goal

Build a routing framework that handles **both** problems at once:

- ① **Stochastic travel times** — treat edge weights as probability distributions
- ② **Congestion-aware assignment** — spread drivers across different routes to avoid overloading any single road

Two Levels of Reliability

System level — α

“With probability $\geq \alpha$, **no road** is overloaded.”

User level — β

“With probability $\geq \beta$, **each driver** arrives on time.”

Key question: What is the best β^* for a given α ?

Can we find it *analytically*?

Where our essay Stands

Gap

Previous work treats **stochasticity** and **congestion** separately. Few has combined them into one framework.

We review two lines of related work:

- ① Deterministic congestion-aware routing
- ② Stochastic shortest-path methods

Outline

- 1 Introduction
- 2 Methodology**
- 3 Implementation & Results
- 4 Discussion & Conclusion

Setup

- Road network $G = (V, E)$; each edge e has $W_e \sim \mathcal{N}(\mu_e, \sigma_e^2)$
- Time discretized into steps $\tau \in \{1, \dots, T_{\max}\}$
- Query $q = (s_q, d_q, t_q)$: origin, destination, departure time
- Path $p = (e_1, \dots, e_k)$ has total travel time $W_p \sim \mathcal{N}(\mu_p, \sigma_p^2)$

Because travel times are random, we **don't know exactly** where a car is at any given time step.

⇒ This leads to our first key concept ...

Def 1: Probabilistic Occupancy

Definition (Probabilistic Occupancy)

$\pi_{p,q}(e, \tau) \in [0, 1]$ is the probability that a vehicle (query q , path p) is on edge e at time τ .

Let $A_p^{(i)} = t_q + \sum_{j=1}^{i-1} W_{e_j}$ be the arrival time at edge e_i . Then:

$$\pi_{p,q}(e_i, \tau) = \Pr[A_p^{(i)} \leq \tau < A_p^{(i)} + W_{e_i}]$$

Computed from Gaussian CDFs of partial sums.

For any fixed τ : $\sum_i \pi(e_i, \tau) \leq 1$ (car is on at most one edge)

Example 1: Probabilistic Occupancy

Example

Path $p = (e_1, e_2)$ with $W_{e_1} \sim \mathcal{N}(5, 1)$, $W_{e_2} \sim \mathcal{N}(3, 1)$, departing at $\tau = 0$.

At $\tau = 4$: $\pi(e_1, 4) = \Pr[W_{e_1} > 4] \approx 0.84$

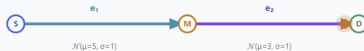
At $\tau = 10$: $\pi(e_2, 10) \approx 0$ (trip almost surely done)

Deterministic model: hard switch at $\tau = 5$, then at $\tau = 8$

Our model: smooth, probabilistic transitions

Example 1: Probabilistic Occupancy

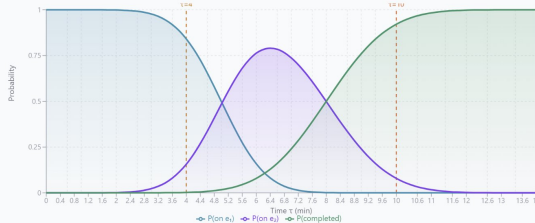
PATH STRUCTURE

Edge Probabilities $\pi(e, \tau)$

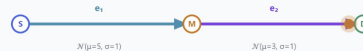
Stochastic vs Deterministic

Travel Time PDFs

LOCATION PROBABILITY OVER TIME

 $\pi_{\mu, \sigma}(e, \tau)$ — Probability the vehicle is on each edge at time τ 

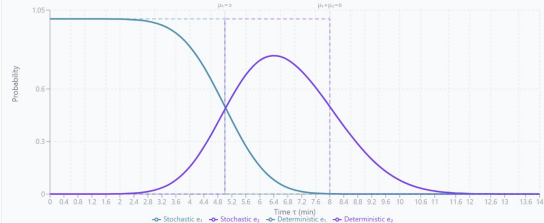
PATH STRUCTURE

Edge Probabilities $\pi(e, \tau)$

Stochastic vs Deterministic

Travel Time PDFs

STOCHASTIC VS DETERMINISTIC MODEL

Solid = stochastic (smooth transitions) - Dashed = deterministic (hard switches at $\tau=5$ and $\tau=8$)

Def 2: α -Reliable System Volume

Definition (α -Reliable System Volume)

The upper α -quantile of aggregate traffic on edge e at time τ :

$$\hat{v}(e, \tau, \alpha) = \mu_{\text{sys}}(e, \tau) + Z_{\alpha} \cdot \sigma_{\text{sys}}(e, \tau)$$

Each query contributes a Bernoulli variable $X_q \sim \text{Bern}(\pi_{f(q)}(e, \tau))$. Sum $\approx$ Normal (Central Limit Theorem):

$$\mu_{\text{sys}} = \sum_q \pi_{f(q)}, \quad \sigma_{\text{sys}}^2 = \sum_q \pi_{f(q)}(1 - \pi_{f(q)})$$

Higher $\alpha \Rightarrow$ more conservative estimate $\Rightarrow$ safer guarantee

Example: 100 queries each with $\pi = 0.05$ give $\mu = 5.0$, $\sigma \approx 2.18$, so at $\alpha = 0.95$:
 $\hat{v} \approx 8.59$ — “95% confident ≤ 9 cars on this edge.”

Def 3: β -Feasible Path Set

Definition (β -Feasible Path Set)

The β -reliable travel budget of path p :

$$\hat{c}(p, \beta) = \mu_p + \Phi^{-1}(\beta) \cdot \sigma_p$$

The feasible set for query q (with detour factor $\delta \geq 0$):

$$\mathcal{F}(q, \beta) = \{p \in \mathcal{P}_q \mid \hat{c}(p, \beta) \leq (1 + \delta) \hat{c}(p_q^*, \beta)\}$$

Here δ is the **detour factor**: a path is β -feasible if its reliable travel time is no more than $(1 + \delta)$ times the *optimal* reliable travel time $\hat{c}(p_q^*, \beta)$.

Higher $\beta \Rightarrow$ high-variance paths get penalized $\Rightarrow$ only **predictable** paths survive $\Rightarrow$ tighter occupancy profiles

Example 3: β -Feasible Paths

Example

Two paths:

$$p_1: (\mu, \sigma) = (10, 4)$$

$$p_2: (\mu, \sigma) = (12, 1)$$

	$\hat{c}(p_1)$	$\hat{c}(p_2)$
$\beta = 0.5 \ (Z = 0)$	10.00	12.00
$\beta = 0.95 \ (Z = 1.645)$	16.58	13.65

At low β : faster mean wins.

At high β : lower variance wins.

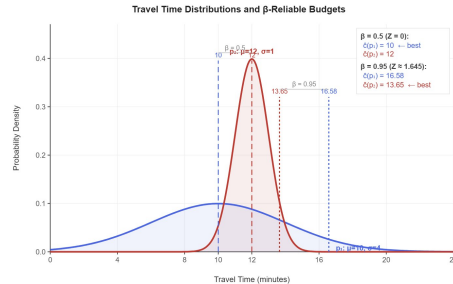


Figure 3 (Def. 3 – Example). Travel-time distributions for p_1 ($\mu = 10, \sigma = 4$) and p_2 ($\mu = 12, \sigma = 1$). Vertical lines mark the β -reliable budgets $\hat{c}(p, \beta)$. At $\beta = 0.5$ the budget equals the mean (p_1 preferred); at $\beta = 0.95$ the variance penalty reverses the ranking in favour of the low-variance p_2 .

Def 4: The Optimization Problem

Definition (Reliability-Constrained Congestion Minimization)

Find β^* and assignment f^* that minimize worst-case load:

$$\min_{\beta} \left(\min_f \max_{e, \tau} \frac{\hat{v}(e, \tau, \alpha)}{c(e)} \right) \quad \text{where} \quad f(q) \in \mathcal{F}(q, \beta) \quad \forall q$$

The trade-off:

- $\beta \uparrow \Rightarrow$ fewer feasible paths (less flexibility)
- $\beta \uparrow \Rightarrow$ sharper occupancy (lower variance)

Closed-Form β^* : Linear Route Choice Model

Why a linear relaxation?

- Step function $\Rightarrow$ not differentiable
- Softmax $\Rightarrow$ breaks Gaussianity

Linear Weight Assignment

Let $Z_\beta = \Phi^{-1}(\beta)$. Assign path p a weight via a **linear penalty** relative to the average candidate cost:

$$W_p(Z_\beta) = \frac{1}{|\mathcal{P}_q|} - \theta(\hat{c}_p(Z_\beta) - \bar{c}(Z_\beta)) = W_p(0) - \theta Z_\beta(\sigma_p - \bar{\sigma}_q)$$

$\theta > 0$: user sensitivity; $\bar{\sigma}_q$: avg. std. dev. of candidate paths

Linearity in Z_β ensures weighted path costs remain **Gaussian**, by applying Central Limit Theorem (CLT).

System Load as a Linear Function of Z_β

At the bottleneck (e^*, τ^*) , the expected system load is:

$$\mu_{\text{sys}}(Z_\beta) = \sum_{q \in Q} \sum_{p \in \mathcal{P}_q} W_p(Z_\beta) \pi_p(e^*, \tau^*)$$

Substituting the linear form of $W_p(Z_\beta)$:

$$\mu_{\text{sys}}(Z_\beta) = \mu_{\text{sys}}(0) - \theta Z_\beta \Omega_\mu$$

Mean Sensitivity Coefficient

$$\Omega_\mu = \sum_{q \in Q} \sum_{p \in \mathcal{P}_q} \pi_p(e^*, \tau^*) \cdot (\sigma_p - \bar{\sigma}_q)$$

System Variance as a Linear Function of Z_β

Each occupancy is Bernoulli-like, so the variance contribution of a path is proportional to $\pi_p(e^*, \tau^*)(1 - \pi_p(e^*, \tau^*))$.

With the same linear-weight argument:

$$\sigma_{\text{sys}}^2(Z_\beta) = \sigma_{\text{sys}}^2(0) - \theta Z_\beta \Omega_v$$

Variance Sensitivity Coefficient

$$\Omega_v = \sum_{q \in Q} \sum_{p \in \mathcal{P}_q} \pi_p(e^*, \tau^*)(1 - \pi_p(e^*, \tau^*)) \cdot (\sigma_p - \bar{\sigma}_q)$$

Result: Both μ_{sys} and σ_{sys}^2 are linear in Z_β .

Deriving the Optimal β^* (1/2)

The system reliable volume objective at the bottleneck:

$$\Lambda(Z_\beta) = \mu_{\text{sys}}(Z_\beta) + Z_\alpha \sqrt{\sigma_{\text{sys}}^2(Z_\beta)}$$

To minimize congestion, we set $\frac{\partial \Lambda}{\partial Z_\beta} = 0$.

This yields the optimal system standard deviation:

Optimal System Standard Deviation

$$\sigma_{\text{sys}}^* = -\frac{Z_\alpha \cdot \Omega_v}{2 \Omega_\mu}$$

Deriving the Optimal β^* (2/2)

Since $\sigma_{\text{sys}}^2(Z_\beta) = \sigma_{\text{sys}}^2(0) - \theta Z_\beta \Omega_v$, we substitute the optimal σ_{sys}^* and solve for Z_β :

Closed-Form β^*

$$Z_{\beta^*} = \frac{\sigma_{\text{sys}}^2(0) - (\sigma_{\text{sys}}^*)^2}{\theta \cdot \Omega_v}$$
$$\Rightarrow \boxed{\beta^* = \Phi(Z_{\beta^*})}$$

Advantage: Recovered in $O(1)$ time — **no iterative search needed.**

RS-SOR: Problem and State

Route queries Q online on $G = (V, E)$, stochastic weights $W_e \sim \mathcal{N}(\mu_e, \sigma_e^2)$.

Goal: minimise $\Lambda = \max_{e, \tau} \frac{\hat{v}(e, \tau, \alpha)}{c(e)}$, where $\hat{v}(e, \tau, \alpha) = \mu(e, \tau) + Z_\alpha \sqrt{\sigma^2(e, \tau)}$

is the α -quantile reliable load — stricter than SOR's integer count.

Maintain per edge-timeslot (e, τ) :

- $\mu(e, \tau), \sigma^2(e, \tau)$ — accumulated expected load and variance
- $x(e, \tau) \propto \exp\left(\frac{\mu(e, \tau)}{\lambda c(e)}\right)$ — congestion penalty, init. $\frac{1}{2mTc(e)}$

High load $\Rightarrow$ exponentially higher $x \Rightarrow$ future queries naturally avoid congested edges.

$\lambda \leftarrow \min_e \frac{1}{c(e)}$ (running estimate of Λ^*), β^* pre-computed offline (Algorithm 1).

Per-Query Routing: Candidate Set and Path Selection

Step 1 — Build candidate set $\mathcal{P}_{\text{cand}}(q)$:

$$\hat{c}(p, \beta^*) = \mu_p + \Phi^{-1}(\beta^*) \cdot \sigma_p \leq (1 + \delta) \cdot C_{\min}$$

$\hat{c}(p, \beta^*)$ is the β^* -quantile travel time of path p . Candidate paths must be both *fast* and *reliable*. NRP pruning discards any DFS branch that cannot possibly satisfy this bound.

Step 2 — Select the path minimising current system congestion:

$$p^* = \arg \min_{p \in \mathcal{P}_{\text{cand}}(q)} \sum_{(e, \tau) \in p} \pi_{p, q}(e, \tau) \cdot x(e, \tau)$$

What is $\pi_{p, q}(e, \tau)$? Because travel times are stochastic, a vehicle departing at τ_0 arrives at edge e at an *uncertain* time. $\pi_{p, q}(e, \tau) \in [0, 1]$ is the probability that the vehicle is actually on edge e at timeslot τ . Routing minimises the *expected* congestion contribution, not a single deterministic assignment.

Per-Query Routing: State Update

After selecting p^* , update every traversed (e, τ) :

Load statistics:

$$\mu(e, \tau) \ += \ \pi_{p^*}(e, \tau), \quad \sigma^2(e, \tau) \ += \ \pi_{p^*}(e, \tau)(1 - \pi_{p^*}(e, \tau))$$

Congestion penalty:

$$x(e, \tau) \ \times = \ \left(1 + \frac{1}{2\lambda c(e)}\right)^{\pi_{p^*}(e, \tau)}$$

This multiplicative form keeps $x \propto \exp\left(\frac{\mu(e, \tau)}{\lambda c(e)}\right)$ *incrementally* after each query — no full recompute over all (e, τ) is needed.

The next query will see updated x values and therefore be steered away from edges that have just accumulated load.

Penalty Adjustment: The Doubling Trick

Trigger — checked after selecting p^* , before updating:

$$\exists (e, \tau) \in p^* : x(e, \tau) > \frac{e^{1/2}}{c(e)} \quad \text{or} \quad \sum_{(e, \tau) \in p^*} \pi_{p^*}(e, \tau) \cdot x(e, \tau) > \lambda$$

Either condition signals that x is growing out of a meaningful range, i.e., λ **is too small**.

Response:

- ① $\lambda \leftarrow 2\lambda$
- ② Recompute $x(e, \tau) \leftarrow \frac{1}{2mTc(e)} \exp\left(\frac{\mu(e, \tau)}{\lambda c(e)}\right)$ for all (e, τ)
- ③ Re-select p^* under the new weights

Accumulated μ, σ^2 are **preserved** — only the global scale λ changes, which rescales all x back into a valid range without discarding routing history.

$O(\ln n)$ -Competitive Guarantee

Two facts chain together:

(1) Bounded $x \Rightarrow$ bounded reliable load.

The trigger keeps $x(e, \tau) \leq \frac{e^{1/2}}{c(e)}$ at all times, so $\frac{\mu(e, \tau)}{\lambda c(e)} \leq \frac{1}{2}$. Substituting into $x \propto e^{\mu/\lambda c}$ and taking logarithms:

$$\frac{\hat{v}(e, \tau, \alpha)}{c(e)} = O(\lambda \ln n)$$

(2) Trigger fires $\Rightarrow \lambda$ was below the optimum.

A primal-dual argument shows: if either trigger condition holds, the current λ cannot support any feasible routing, so $\lambda < \Lambda^*$. After at most $O(\log(1/\varepsilon))$ doublings, the final $\lambda < 2\Lambda^*$.

Combining (1) and (2):

$$\Lambda = O(\lambda \ln n) = O(\Lambda^* \cdot \ln n)$$

Outline

- 1 Introduction
- 2 Methodology
- 3 Implementation & Results**
- 4 Discussion & Conclusion

Data Source

Data

- **NYC TLC:** Yellow Taxi, Green Taxi, For-Hire Vehicle (FHV), High Volume FHV — 15 months of records
- 42 Parquet files → 263 zones, 66,099 directed edges

Each TLC record contains: pickup zone, drop-off zone, and trip duration.

We treat each unique (pickup, drop-off) pair as a directed edge and compute per-edge μ_e and σ_e via `groupby()`.



Data Challenges

Challenge 1: Spatial Granularity

TLC records trips between **large macro zones** (e.g., Area A to Area B), not microscopic intersections. Detours are treated as entirely separate zone-to-zone trips
⇒ inflated travel times and rigid routing.

Challenge 2: DIMACS Incompatibility

The DIMACS road network offers node-level topology, but only **deterministic** (static) edge weights. Our framework requires **stochastic** distributions (μ, σ). A reliable spatial mapping between TLC zones and DIMACS nodes proved infeasible.

Data Solution: Massive Scale-Up (1/2)

Key Hypothesis

Massively scaling up TLC data implicitly reveals smaller-scale routing steps within the zone graph.

Expansion Details:

- Expanded to **all 2025** NYC TLC records **plus** available 2026 data
- All vehicle types included: Yellow Taxi, Green Taxi, FHV, HVFHV
- Dense Origin-Destination matrix $\Rightarrow$ trips like “A $\rightarrow$ B” can now be constructed from “A $\rightarrow$ C, C $\rightarrow$ D, D $\rightarrow$ B”

Data Solution: Massive Scale-Up (2/2)

This scale-up provided the comprehensive, finer-grained stochastic information needed, drastically improving routing decisions.

Processing Pipeline:

- Standardize columns
- Filter trips ($0 < \text{dur} < 180 \text{ min}$)
- Remove zero-distance & same-zone trips
- Compute per-pair (μ_e, σ_e)



Evaluation Methodology (1/2)

Query Generation

From a set-aside month: 1,000 consecutive trip records as test queries, providing original origin, destination, and departure time. Fed in **chronological order** (as if an online arrival model).

Evaluation Axis 1: User-level (Travel Time)

- Compare the algorithm's predicted travel time vs. historical mean duration.
- Flag if the ratio exceeds our detour threshold (we set it to be 1.5 in our program).

That "1.5" threshold is actually feasible (1/2)

Real-World Navigation Reality Check



Prediction vs Reality

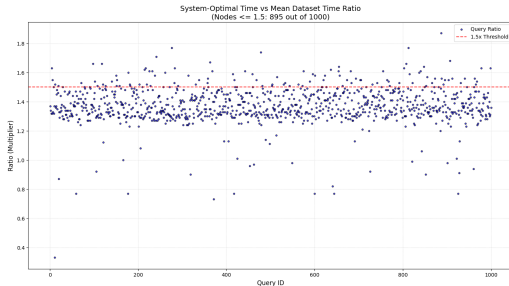
The actual driving time was **1.58**× the originally predicted time.

Significant delays are common in real-world urban navigation.

That "1.5" threshold is actually feasible (2/2)

- **Prediction vs Reality:** In the previous example, the actual time was **1.58**× the predicted time.
- Commercial apps (like Google Maps or Amap) often impose similar or larger re-routing overheads due to real-time traffic dynamics.
- **Conclusion:** Imposing a $1.5\times$ **threshold** ($(1 + \delta) = 1.5$) for detour allowance is generous, realistic, and practically acceptable for users.

User-Level Results: Travel Time Scatter Plot



- **89.5%** stay within $1.5\times$ of historical mean travel time.
- Most cluster between $1.2\times$ and $1.5\times$.
- About **10.5%** of queries go above $1.5\times$.

Reasons:

- Noisy TLC records (GPS errors, wrong timestamps)
- Sparse OD pairs with unreliable mean estimates
- Genuine re-routing away from critical bottlenecks

Evaluation Methodology (2/2)

Evaluation Axis 2: System-level (Congestion Reduction)

Maximum Reliable Load Metric:

$$\Lambda = \max_{e, \tau} \frac{\hat{v}(e, \tau, \alpha)}{c(e)}$$

Baseline Comparison:

Compare RS-SOR assignments versus a classic **Shortest Path baseline** under identical queries and stochastic conditions.

System-Level: Congestion Reduction

$\alpha = 0.95$, $\theta = 0.1$, $K = 5$, $\delta = 0.3$, NYC Yellow Taxi Jan 2026

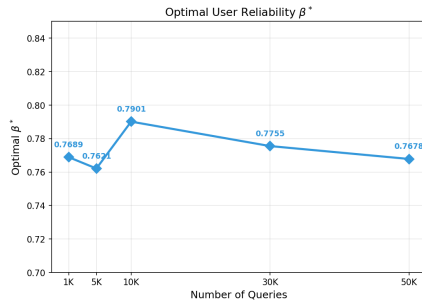
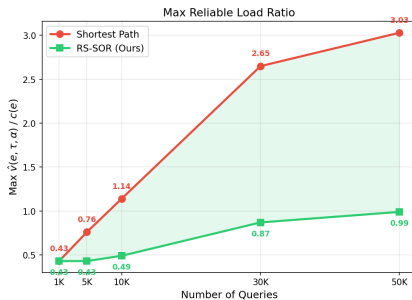
Queries	Shortest Path	RS-SOR	Reduction	β^*
1K	0.43	0.43	1.00×	0.769
5K	0.76	0.43	1.77×	0.762
10K	1.14	0.49	2.33×	0.790
30K	2.65	0.87	3.05×	0.776
50K	3.03	0.99	3.06×	0.768

RS-SOR keeps max load below 1.0 for all workloads tested.

System-Level: Congestion Reduction

RS-SOR Experiments ($\alpha=0.95$, $\theta=0.1$, $K=5$, $\delta=0.3$, NYC Yellow Taxi 2026-01)

RS-SOR Experiments ($\alpha=0.95$, $\theta=0.1$, $K=5$, $\delta=0.3$, NYC Yellow Taxi 2026-01)



Outline

- 1 Introduction
- 2 Methodology
- 3 Implementation & Results
- 4 Discussion & Conclusion**

Discussion: Limitations

Limitations

- **Distributional assumption:** Normal distribution may not fit **skewed** real traffic (e.g., heavy-tailed delays from accidents).
- **Spatial granularity:** 263 TLC zones is **coarse** compared to street-level networks (DIMACS NYC has 264K nodes, 734K edges).
- **Scalability:** Single-threaded implementation; becomes slow past a few thousand queries. Not yet suitable for real-time city-scale deployment.

Conclusion (1/2)

Three Main Contributions

- 1 **Unified Formulation:** Combined system reliability α and user reliability β in a single stochastic routing framework.
- 2 **Analytical Solution:** Derived closed-form β^* via the Linear Route Choice Model — completely eliminating iterative search.
- 3 **RS-SOR Algorithm:** Achieved up to $3\times$ congestion reduction, while keeping $\sim 90\%$ of users within acceptable detour limits ($1.5\times$).

Conclusion (2/2)

Directions for Future Work

- **Network Integration:** Implement graph mapping to test on street-level topological graphs (e.g., DIMACS NYC).
- **Advanced Distributions:** Extend the framework to handle Non-Gaussian travel time distributions (like log-normal or heavy-tailed distributions).
- **System Engineering:** Develop a parallelized, GPU-accelerated implementation for real-time, city-scale deployment.

Thank you for listening!

Yichen Ren & Yixuan Fu

Advised by Prof. Raymond Chi Wing Wong

Department of Computer Science
The Hong Kong University of Science and Technology

COMP 4981 Final Year Project ■ 2025–2026